

SKILL_WEEK-05

(SESSION NUMBER - 9)

Task 1: To Change directories, validate paths, update the working directory, handle errors, maintain the previous directory, and test navigation.

```
UW PICO 5.09

    {
        perror("chdir failed");
    }
}
else
{
    char temp[PATH_SIZE];

    if (getcwd(temp, sizeof(temp)) == NULL)
    {
        perror("getcwd failed");
        continue;
    }

    if (chdir(path) == 0)
    {
        strcpy(previous, temp);

        getcwd(current, sizeof(current));
        printf("Current Directory: %s\n", current);
    }
    else
    {
        perror("Invalid directory");
    }
}

return 0;
}
```

Fig 1: C program for directory navigation, path validation, and maintaining the previous directory

```
amrutha@Amruthas-MacBook-Air OSSP % ./bin/skill9_task1
Initial Directory: /Users/amrutha/OSSP

Enter directory path (or '-' for previous, 'exit' to stop): ..
Current Directory: /Users/amrutha

Enter directory path (or '-' for previous, 'exit' to stop):
```

Fig 2: Successful directory change using a valid relative path

```
Enter directory path (or '-' for previous, 'exit' to stop): -
Changed to previous directory: /Users/amrutha/OSSP
```

Fig 3: Returning to the previous working directory using stored directory state

```
Enter directory path (or '-' for previous, 'exit' to stop): /not/a/real/path
Invalid directory: No such file or directory
```

Fig 4: Handling an invalid directory path during navigation

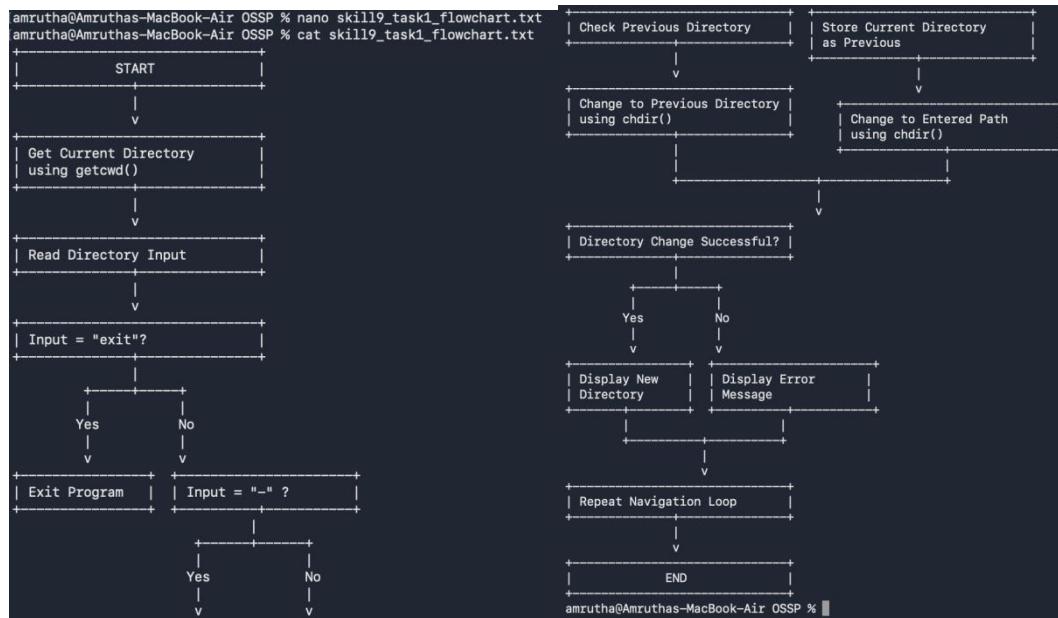


Fig 5: Flowchart for directory navigation and previous-directory management

Result:

Directory navigation was implemented successfully using `chdir()` and `getcwd()`. Valid paths were handled correctly, invalid paths produced appropriate errors, the current working directory was updated, and the previous directory was maintained for backward navigation.

Task 2: Identify built-ins, create a dispatch table, execute in-process commands, handle invalid commands, maintain state, and test dispatch logic.

```
UW PICO 5.09

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>

#define MAX_INPUT 256

int builtin_pwd(char *arg)
{
    char cwd[1024];

    if (getcwd(cwd, sizeof(cwd)) != NULL)
    {
        printf("Current Directory: %s\n", cwd);
    }
    else
    {
        perror("getcwd failed");
    }

    return 1;
}

int builtin_cd(char *arg)
{
    if (arg == NULL)
    {
        printf("cd: directory required\n");
        return 1;
    }

    int builtin_cd(char *arg)
    {
        if (arg == NULL)
        {
            printf("cd: directory required\n");
            return 1;
        }

        if (chdir(arg) == 0)
        {
            printf("Directory changed successfully.\n");
        }
        else
        {
            perror("cd failed");
        }

        return 1;
    }

    int builtin_help(char *arg)
    {
        printf("Available Built-in Commands:\n");
        printf("pwd      - Display current directory\n");
        printf("cd <path> - Change directory\n");
        printf("help     - Display help\n");
        printf("exit     - Exit mini shell\n");

        return 1;
    }

    int builtin_exit(char *arg)
    {
        if (arg == NULL)
        {
            printf("exit: no argument required\n");
            return 1;
        }

        return 1;
    }
}
```

Fig 6: C program implementing built-in commands using a dispatch table and function pointers

```
[amrutha@Amruthas-MacBook-Air OSSP % gcc src/skill9_task2.c -o bin/skill9_task2]
[amrutha@Amruthas-MacBook-Air OSSP % ./bin/skill9_task2]

mini-shell> pwd
Current Directory: /Users/amrutha/OSSP
```

Fig 7: Executing the built-in pwd command through the dispatch table

```
mini-shell> help
Available Built-in Commands:
pwd      - Display current directory
cd <path> - Change directory
help     - Display help
exit     - Exit mini shell
```

Fig 8: Displaying supported built-in commands through the dispatch table

```
mini-shell> cd ..
Directory changed successfully.

mini-shell> pwd
Current Directory: /Users/amrutha
```

Fig 9: Maintaining shell state after executing the built-in cd command

```
mini-shell> hello
Invalid built-in command: hello
```

Fig 10: Handling an invalid command during dispatch-table lookup

```
mini-shell> exit
Exiting mini shell...
```

Fig 11: Exiting the mini shell using the built-in exit command



Fig 12: Flowchart for executing built-in commands using a dispatch table

Result:

Built-in commands were identified and executed successfully using a dispatch table and function pointers. Commands such as `pwd`, `cd`, `help`, and `exit` were handled within the same process, shell state was maintained, and invalid commands were detected correctly.

(SESSION NUMBER - 10)

Task 1: To Retrieve the current directory, display the path, process exit requests, clean up resources, save state, and verify correct behavior.



```
UW PICO 5.09

    break;
}

if (strcmp(command, "exit") == 0)
{
    printf("\nExit request received.\n");

    state_file = fopen("skill10_state.txt", "w");

    if (state_file != NULL)
    {
        fprintf(state_file,
            "Last Working Directory: %s\n",
            cwd);

        fclose(state_file);

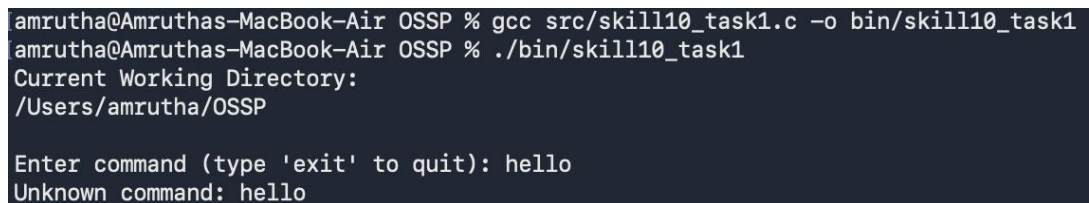
        printf("Program state saved successfully.\n");
    }
    else
    {
        printf("Unable to save state.\n");
    }

    printf("Cleaning up resources...\n");
    printf("Program terminated successfully.\n");

    break;
}
else
{
    printf("Unknown command: %s\n", command);
}

return 0;
}
```

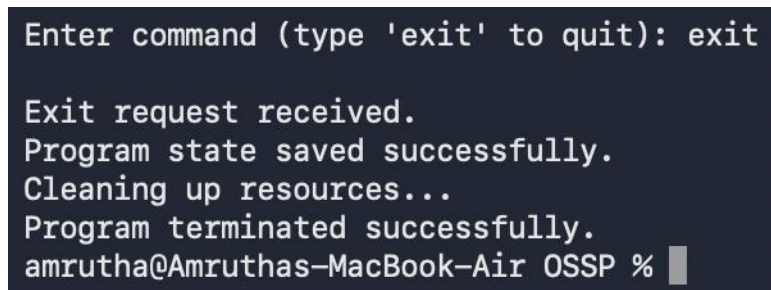
Fig 1: C program for retrieving the current directory, handling exit requests, saving state, and performing cleanup



```
amrutha@Amruthas-MacBook-Air OSSP % gcc src/skill10_task1.c -o bin/skill10_task1
amrutha@Amruthas-MacBook-Air OSSP % ./bin/skill10_task1
Current Working Directory:
/Users/amrutha/OSSP

Enter command (type 'exit' to quit): hello
Unknown command: hello
```

Fig 2: Handling an unknown command while the program remains active



```
Enter command (type 'exit' to quit): exit

Exit request received.
Program state saved successfully.
Cleaning up resources...
Program terminated successfully.
amrutha@Amruthas-MacBook-Air OSSP %
```

Fig 3: Processing the exit request, saving program state, and cleaning up resources

```

amrutha@Amruthas-MacBook-Air OSSP % cat skill10_state.txt
Last Working Directory: /Users/amrutha/OSSP
amrutha@Amruthas-MacBook-Air OSSP %

```

Fig 4: Verification of the saved program state after graceful termination

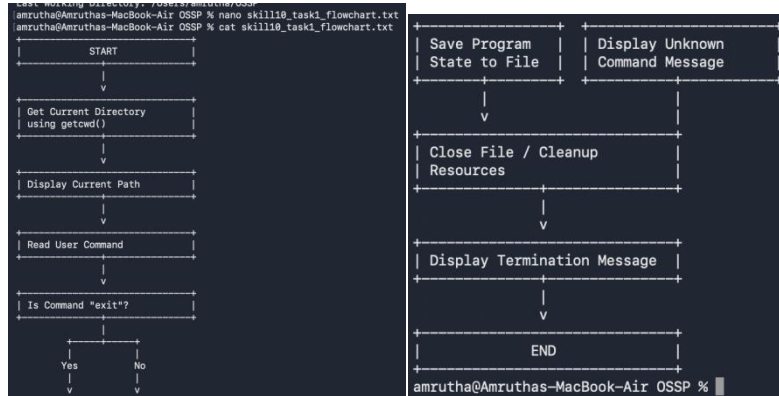


Fig 5: Flowchart for retrieving the current directory, saving state, and handling graceful program termination

Result:

The current working directory was retrieved and displayed successfully. Exit requests were processed correctly, the program state was saved to a file, resources were cleaned up, and the program terminated gracefully.

Task 2: Parse export syntax, update environment variables, validate names, handle existing variables, export to child processes, and test functionality.

```

LW PICO 5.09
}
return 1;
}
if (setenv(name, value, 1) != 0)
{
    perror("setenv failed");
    return 1;
}
printf("Environment variable updated successfully.\n");
printf("%s = %s\n", name, getenv(name));
pid_t pid = fork();
if (pid < 0)
{
    perror("fork failed");
    return 1;
}
if (pid == 0)
{
    printf("\nChild Process:\n");
    printf("Inherited %s = %s\n", name, getenv(name));
    exit(0);
}
else
{
    wait(NULL);
    printf("\nParent Process:\n");
    printf("Export verification completed.\n");
}
return 0;
}

```

Fig 6: C program for parsing export syntax and updating environment variables

Test Case 1:

```
amrutha@Amruthas-MacBook-Air OSSP % ./bin/skill10_task2
Enter export command: export COURSE=OSSP
Environment variable updated successfully.
COURSE = OSSP

Child Process:
Inherited COURSE = OSSP

Parent Process:
Export verification completed.
```

Fig 7: Successful export of an environment variable and inheritance by the child process

Test Case 2:

```
amrutha@Amruthas-MacBook-Air OSSP % ./bin/skill10_task2
Enter export command: export COURSE=OperatingSystems
Environment variable updated successfully.
COURSE = OperatingSystems

Child Process:
Inherited COURSE = OperatingSystems

Parent Process:
Export verification completed.
```

Fig 8: Updating an existing environment variable using export syntax

Test Case 3:

```
amrutha@Amruthas-MacBook-Air OSSP % ./bin/skill10_task2
Enter export command: export 1COURSE=OSSP
Invalid variable name: 1COURSE
amrutha@Amruthas-MacBook-Air OSSP %
```

Fig 9: Detection of an invalid environment variable name

Test Case 4:

```
amrutha@Amruthas-MacBook-Air OSSP % ./bin/skill10_task2
Enter export command: export COURSE
Invalid syntax. '=' is missing.
amrutha@Amruthas-MacBook-Air OSSP %
```

Fig 10: Handling invalid export syntax with a missing assignment operator



Fig 11: Flowchart for parsing export syntax, updating environment variables, and verifying child-process inheritance

Result:

Export syntax was parsed successfully, valid environment variable names were accepted, existing variables were updated using `setenv()`, invalid inputs were handled correctly, and exported variables were inherited by the child process.